Quantum: Privacy-Preserving Post-Quantum DAG Protocol

Research Design Draft 0.4.0-research and Security Requirements

Phexora AI · phexora.ai

2026-07-23

Abstract

Quantum is a research design for a private note-based DAG protocol intended to combine post-quantum security, privacy and network anonymity by default, and at least 1,000 accepted layer-1 transactions per second. This manuscript defines the non-negotiable requirements, threat and claim boundaries, candidate architecture, and blocking evidence gates. It does not report a conformant implementation, completed security proof, external audit, testnet, or production-ready protocol. The initial product boundary is deliberately narrow: private post-quantum digital cash and settlement with bounded payment policies, not a general-purpose smart-contract platform.

Keywords: post-quantum cryptography; privacy-preserving ledger; DAG consensus; zero-knowledge proofs; protocol research.

Document status

This document defines the non-negotiable security and scalability requirements for Quantum and a candidate architecture for meeting them. It is a **research design draft**, not a completed protocol specification. There is no conformant implementation, testnet, security proof, external audit, or production network at this revision.

The legacy filename contains "v1" so that existing links remain valid. The normative document version is 0.4.0-research.

The words **MUST**, **MUST NOT**, **REQUIRED**, **SHOULD**, and **MAY** describe research acceptance criteria. They do not imply that an implementation already satisfies them.

Non-negotiable product requirements

Quantum is releasable only if all three properties hold together:

1. **Post-quantum security**: every security-critical primitive and their composition meet an end-to-end post-quantum security target of at least 128 bits against quantum polynomial-time adversaries.
2. **Privacy and anonymity by default**: the ledger has no transparent transaction mode; sender, recipient, amount, and transaction-graph relationships are hidden within a declared threat model, and the network layer does not expose a simple transaction-to-origin mapping.
3. **Scalability**: the complete protocol, including proof generation, verification, state updates, networking, and storage, sustains at least 1,000 accepted layer-1 transactions per second under a published, reproducible workload.

If any requirement cannot be met, the design MUST be changed or the project MUST stop before a production launch. Classical cryptographic fallbacks, optional transparent transactions, and benchmark shortcuts are not permitted.

1. Scope and claim boundary

This document covers:

- a private note-based UTXO transaction model;
- post-quantum authorization and recipient encryption;

- transparent zero-knowledge validity proofs;
- deterministic DAG consensus and state ordering;
- network-layer anonymity requirements;
- supply integrity, rewards, serialization, and resource limits;
- the evidence required before testnet or production claims.

This document does not claim that combining standardized components automatically produces a secure protocol. Composition, implementation, side channels, consensus integration, and network metadata require separate analysis.

1.1 Initial product boundary and sequencing

Quantum's initial product target is **private post-quantum digital cash and settlement** for people, organisations, and autonomous software agents. The core profile consists of native-QTM transfers, private rewards, wallet recovery, selective user-controlled disclosure, and the smallest deterministic payment-policy surface that passes the same privacy, supply, and performance gates as an ordinary transfer.

The initial profile is not a general-purpose application platform. Arbitrary smart contracts, an EVM/SVM-compatible runtime, general DeFi state, permissionless asset issuance, bridges, and cross-chain message execution are outside the core release scope. They MUST NOT be assumed by the security proof or used to defer a core release requirement. Any later capability requires a separately versioned threat model, validity relation, resource budget, and release decision.

Research MUST proceed through explicit stop/go boundaries:

1. freeze the threat model, encodings, commitment candidate, and proof profile;
2. implement and benchmark the complete representative private transaction described in Section 9.3;
3. stop or revise the cryptographic profile if that feasibility gate fails;
4. only then integrate wallets, private state, DAG consensus, transport, and full-node operation;
5. add bounded payment policies or interoperability adapters only after their additional proof and trust assumptions pass separate gates.

This sequencing does not weaken the three joint production requirements. It prevents node and ecosystem work from concealing a cryptographic design that cannot meet them.

1.2 Status vocabulary

Status	Meaning
Requirement	Mandatory property of any release
Selected standard	Exact external standard selected; integration still requires validation
Candidate	Design direction that may change after analysis
Blocking research gate	No implementation or release may rely on this area until its deliverables pass
Verified	Reserved for independently reproducible evidence; no subsystem has this status yet

1.3 Current subsystem status

Subsystem	Current status	Required next evidence
SHAKE256	Selected standard: FIPS 202	Domain-separation vectors and implementation KATs

Subsystem	Current status	Required next evidence
Spend authorisation	Comparative research gate; stateless incumbent: FIPS 205, SLH-DSA-SHAKE-256f	Stateful/stateless comparison, KATs, state-failure analysis, side-channel review, proof-system cost
ML-KEM	Selected standard: FIPS 203, ML-KEM-1024	FIPS KATs and an authenticated composition
Note commitment	Blocking research gate	Exact construction, parameters, reduction, estimator report, review
Zero-knowledge STARK	Blocking research gate	Exact field/transcript/FRI/masking profile and soundness analysis
Representative transaction proof	Blocking research gate	Complete 2-input/2-output prototype and reproducible feasibility report
DAG consensus/state	Blocking research gate	Exact GHOSTDAG profile, deterministic ordering, DAA, state proof
Network anonymity	Blocking research gate	Exact protocol and analysis against the stated observer
Performance	Blocking research gate	End-to-end prototype and reproducible target benchmark
Economics/genesis	Provisional	DAA-score reward rules, cap proof, canonical genesis bytes

2. Normative requirements

R1 — Private ledger

- R1.1 Every ordinary transfer MUST be private. There MUST be no transparent amount, transparent recipient, or optional transparent transaction type.
- R1.2 Public ledger data MUST NOT reveal the transferred values, recipient addresses, spend authorization keys, the recipient key targeted by an encrypted output, or a direct input-to-output link.
- R1.3 A valid nullifier MUST prevent a second spend without identifying the note commitment from which it was derived.
- R1.4 The proof system MUST establish authorization, note membership, output correctness, and exact integer balance without revealing private witness data.
- R1.5 User-controlled disclosure capabilities MAY reveal selected wallet or transaction data. The profile MUST distinguish incoming-view, full-wallet, transaction-specific, and auditor-scoped disclosure as defined in Section 5.7.
- R1.6 Disclosure MUST be explicit, least-privilege, and non-global. It MUST NOT create a protocol backdoor, mandatory universal view key, or privacy loss for notes and users outside the granted scope.

R2 — Network anonymity

- R2.1 Transaction transport MUST be unlinkable to a sender network endpoint within the approved network threat model.
- R2.2 The protocol MUST address timing, volume, route, retry, and peer-selection metadata. Payload encryption alone is insufficient.
- R2.3 Wallet identity keys, note keys, and transaction authorization keys MUST NOT be reused as P2P identities.

- R2.4 Dandelion++ MAY be evaluated as one layer, but MUST NOT by itself be treated as proof against a global observer. Cover traffic, mixing or onion routing, and active-intersection resistance require explicit evaluation.

"Anonymous" in a release claim means that R1 and R2 have passed for the published adversary model. No protocol can hide activity from a compromised endpoint, a voluntarily disclosed view key, or off-chain information supplied by the user; those boundaries MUST be stated with every claim.

R3 — End-to-end post-quantum security

- R3.1 The minimum composed security target is 128 post-quantum bits after multi-user, multi-target, and protocol-lifetime losses.
- R3.2 Authorization, commitments, proofs, key establishment, hashing, authenticated encryption, randomness generation, storage encryption, and upgrade authentication MUST be included in the analysis.
- R3.3 A classical-only handshake or signature MUST NOT be accepted as a fallback.
- R3.4 Every primitive MUST have an exact algorithm identifier, parameter set, canonical encoding, test-vector source, domain-separation rule, required security property, and output length justified by the composed analysis.
- R3.5 A "generic STARK", "lattice-based commitment", or "hybrid" label is not evidence of post-quantum security.

R4 — Authorization and supply integrity

- R4.1 Every input value used by the balance equation MUST be bound to the opening of an existing note commitment.
- R4.2 Every output value used by the balance equation MUST be bound to the opening of the corresponding new note commitment.
- R4.3 Balance MUST be proved as bounded integer arithmetic. Equality in a finite field alone is forbidden.
- R4.4 Only a consensus-authorized reward transaction MAY create value.
- R4.5 The total issued supply MUST never exceed 21,000,000 QTM, represented in a fixed base unit and checked without floating-point arithmetic.
- R4.6 Fees MUST be accounted for as value transferred from accepted ordinary transactions, not as newly issued value. Only the subsidy portion of an authorized reward transition may increase cumulative issuance.
- R4.7 Reward outputs MUST NOT exceed the sum of the consensus-authorized subsidy and fees collected by the exact canonical state transition. Unclaimed value is burned and MUST NOT become reclaimable through another path.

R5 — Deterministic consensus safety

- R5.1 All honest nodes receiving the same valid DAG and state MUST derive the same ordering, accepted transaction set, difficulty, rewards, and state root.
- R5.2 Consensus calculations MUST use deterministic integer arithmetic. Local wall-clock time and floating-point arithmetic MUST NOT determine a consensus result.
- R5.3 Header-declared difficulty is advisory data only. Validation MUST recompute the expected target from the agreed DAG history and reject any mismatch.
- R5.4 Concurrent nullifier conflicts MUST resolve through one canonical ordering and atomic state transition.

R6 — Scalability

- R6.1 The release target is at least 1,000 accepted layer-1 transactions per second, sustained for at least 24 hours on a geographically distributed testbed.

- R6.2 The benchmark MUST include proof verification, signature checks, consensus ordering, nullifier/state updates, propagation, and rejected conflicts. Prevalidated or empty transactions do not count.
- R6.3 Reference hardware, topology, software revision, workload, proof sizes, bandwidth, storage growth, latency percentiles, and failure rate MUST be published.
- R6.4 Every full validator MUST either process the complete accepted state transition stream at the target rate or rely on a separately proven consensus-secure partitioning design. Parallel block production alone does not reduce validator work.
- R6.5 Pruning and snapshots MAY reduce operational storage, but archival requirements and trustless recovery MUST remain explicit.

R7 — Transparent setup and upgrade safety

- R7.1 Transaction validity MUST NOT depend on a secret trusted-setup artifact.
- R7.2 Proof aggregation or recursion is allowed only if the outer proof preserves the post-quantum and no-secret-setup requirements.
- R7.3 Protocol upgrades MUST be versioned, domain-separated, replay-safe, and authenticated by a post-quantum governance mechanism defined before launch.

R8 — Verifiability

- R8.1 Consensus-critical behavior MUST have canonical byte encodings and cross-implementation vectors.
- R8.2 Security arguments MUST state assumptions, reductions, concrete parameters, and composition losses.
- R8.3 Test results MUST distinguish analytical proof, formal verification, implementation tests, statistical diagnostics, benchmarks, and external review. One category MUST NOT be presented as another.
- R8.4 Production claims require at least two independent implementations, public interoperability vectors, and independent cryptographic and consensus review.
- R8.5 Every capability claim MUST identify its versioned profile and evidence gate. A base-layer proof MUST NOT be presented as evidence for an external bridge, general-purpose application runtime, or regulatory compliance.

3. Threat model

The final security profile MUST define exact advantage games and corruption thresholds. The minimum research model includes:

- a quantum polynomial-time cryptographic adversary with adaptive chosen-message and chosen-ciphertext capabilities where applicable;
- malicious transaction creators, provers, miners, peers, and data providers;
- adaptive network delay, eclipse and Sybil attempts, packet observation, transaction injection, and intersection analysis;
- a global passive network observer for the anonymity target, plus explicitly enumerated active attacks;
- long-term ledger retention and later cryptanalytic improvement;
- recipient-key inference from encrypted-note payloads, including multi-user, chosen-key, chosen-ciphertext, and cross-output correlation attacks;
- crashes, reordering, duplicate delivery, and recovery from snapshots;
- side-channel attackers against wallet and validator implementations.

The consensus profile MUST separately state the assumed adversarial work fraction, network-delay model, liveness conditions, and finality rule. These cannot be inferred from the GHOSTDAG name.

Out of scope for a cryptographic anonymity guarantee are compromised endpoints, malicious operating systems, coerced key disclosure, voluntarily published view keys, and identifying off-chain behavior. Implementations still SHOULD minimize the damage of these events.

4. Cryptographic profile

4.1 Hash and domain separation

The candidate hash primitive is SHAKE256 from NIST FIPS 202.

For research vectors, define:

```
QH(tag, message, output_length) =
    SHAKE256(
        ASCII("QTM-RD-0.2") ||
        LE16(length(tag)) || ASCII(tag) ||
        LE64(length(message)) || message ||
        LE32(output_length),
        output_length
    )
```

Tags MUST be non-empty printable ASCII, unique by purpose, and registered in the final protocol profile. Output length is in bytes. Decoders MUST reject unknown protocol versions instead of guessing a legacy rule.

Research vectors:

Tag	Message hex	Output length	SHAKE256 output hex
test	empty	32	c03ab74639696f42275d889eb3ba7753a4effc
txid	00010203	32	b22782c3a5291412ca5bd8cf85edb47aca9d5
empty-leaf	empty	32	432aa478b2724c19a5ea7b5c17f0c983001de1

These vectors validate this wrapper only; they are not a security proof or a substitute for FIPS 202 KATs. Their 32-byte output is a wrapper test parameter, not an approved consensus digest length. Every collision-dependent use MUST derive its output length from the R3 quantum, multi-target, and protocol-lifetime analysis before that use is frozen, including the time/memory models in generic quantum collision research.

4.2 Transaction authorization

The current stateless candidate is SLH-DSA-SHAKE-256f from NIST FIPS 205. The old name SPHINCS+ describes the design lineage; protocol identifiers and public claims MUST use the standardized name SLH-DSA.

This parameter profile is an incumbent under test, not a completed protocol decision. TzEL already demonstrates the high-level alternative of WOTS-like one-time authorisation under an XMSS-style tree inside a private-payment STARK. T202 and T305 MUST therefore compare the incumbent against an independently specified TzEL-shaped baseline and, if its controlled state and key-generation requirements fit the wallet model, one exact stateful profile from NIST SP 800-208. No TzEL code may be reused without compatible permission and documented legal review.

The signed message MUST be a dedicated transaction authorization digest that binds at least:

- protocol version and chain identifier;
- anchor and state context;
- all nullifiers and output commitments in canonical order;
- encrypted-note digests;
- public fee, expiry, and transaction flags;
- authorisation profile; and

- aggregation mode and any external proof reference.

The final profile MUST pin the FIPS 205 interface, context string, randomness mode, key encoding, signature encoding, KATs, and failure behavior. A profile-specific authorization public key and signature MAY remain private witness data only if the zero-knowledge circuit verifies them and the note commitment binds the authorized key. Operational signing state remains wallet-private and is covered by state-management evidence rather than assumed to be validated by the proof. The cost and soundness of in-proof verification are a blocking gate.

Every stateful comparison profile MUST additionally pin key-index allocation, crash consistency, backup/restore rollback behavior, concurrency, exhaustion, recovery, and state desynchronisation. The stateless profile may be retained only if it meets every frozen system requirement and its pre-registered security, interoperability, or state-management benefit materially justifies its measured overhead. A stateful profile may be selected only through a versioned specification change and independent review; it is not a hidden fallback.

FIPS 205 notes that applications needing message-bound signatures must account for the collision cost of H_msg or apply an appropriate reviewed transformation. The transaction-authorization analysis MUST decide whether that property is required, include quantum and multi-target losses, and either show that the composed R3 target remains satisfied or select a reviewed mitigation.

4.3 Recipient key encapsulation

The selected KEM candidate is ML-KEM-1024 from NIST FIPS 203, including applicable NIST errata and KATs.

ML-KEM is not an authenticated transport protocol. The final note-encryption and P2P profiles MUST specify KEM composition, transcript binding, key derivation, authenticated encryption, nonce rules, replay protection, identity separation, and failure behavior. An invented "Noise" pattern name MUST NOT be used unless a real, reviewed Noise extension defines the same messages and security properties. The published Noise Protocol Framework is Diffie–Hellman based and does not itself define an ML-KEM handshake.

FIPS 203 does not by itself establish receiver anonymity or recipient-key privacy. The note-encryption profile MUST define and meet an explicit game in which a public encrypted output does not reveal which eligible recipient key was used, including under multi-user, chosen-key, chosen-ciphertext, and cross-output correlation attacks. Any scanning tag or trial-decryption shortcut is part of that disclosure analysis. Link authentication for P2P channels and receiver anonymity for note delivery are separate properties; the final profile MUST cite and instantiate a reviewed definition such as the anonymous-KEM property or a stronger application-appropriate game.

4.4 Commitment construction — blocking gate

No commitment scheme is selected at this revision. The previous square-matrix formula and four-limb encoding were removed because they did not establish a binding, hiding, carry-safe, homomorphic commitment.

The required interface is:

```
Setup(security_profile) -> public_parameters
Commit(public_parameters, note_plaintext, randomness) -> commitment
Open(public_parameters, commitment, note_plaintext, randomness) -> boolean
```

The committed note plaintext MUST bind:

- protocol version and asset identifier;
- 64-bit base-unit value;
- spend-authorization key digest;
- nullifier-key digest;
- recipient/diversifier data required by the final address scheme;

- creation-domain data needed to prevent cross-chain or cross-version reuse.

The selected construction MUST provide correctness, quantum computational hiding and binding, canonical encodings, domain separation, unbiased randomness sampling, parameter-generation rules, and concrete security at the composed target. If a Module-LWE/Module-SIS construction such as the BDLOP line of work is selected, the actual construction and parameter analysis—not the family name—must be reviewed. Byte reduction modulo a sampler range is forbidden unless rejection sampling removes bias.

Homomorphism is not required: exact conservation is proved inside the transaction validity relation. If homomorphism is later added, carry semantics and all algebraic assumptions require a separate proof.

4.5 Zero-knowledge proof system — blocking gate

The candidate family is a transparent hash-based STARK, informed by the original STARK work and DEEP-FRI. The final profile MUST pin:

- base and extension fields and their encodings;
- AIR constraints and maximum degrees;
- trace padding and public-input encoding;
- Fiat–Shamir transcript order and every domain tag;
- FRI variant, blowup, query count, grinding, and batching;
- zero-knowledge masking/randomization and leakage analysis;
- rejection sampling and challenge bias rules;
- concrete classical and quantum soundness after composition;
- proof size, verifier memory, and denial-of-service limits.

A 64-bit base field alone cannot deliver a 128-bit soundness claim. A generic STARK is not automatically zero knowledge. Empirical failure-free testing cannot prove a soundness probability. The final analysis MUST reach at least the R3 composed target; the former 2^-100 target is insufficient.

An aggregated block format MUST choose exactly one consensus representation:

1. individual transaction proofs; or
2. an aggregate proof plus authenticated references to transactions whose individual proofs are omitted.

Keeping all individual proofs and adding an aggregate does not reduce network or storage load.

Wallet and block proving are separate roles. A wallet MAY produce an individual transaction proof for admission and propagation. If an aggregate mode is selected, the block producer or dedicated aggregator MUST recursively or otherwise soundly combine admitted proofs, and the consensus block MUST carry the compact aggregate plus authenticated transaction references while omitting the individual proofs it replaces. The aggregate relation MUST bind the exact ordered transaction set, public inputs, pre-state, and post-state; aggregation MUST NOT turn proof generation into a trusted service.

4.6 Entropy, mnemonics, and key derivation

Randomness MUST come from an operating-system CSPRNG and MUST fail closed if entropy acquisition fails. Test seeds MUST never be accepted by production builds.

If mnemonic import/export is supported, it MUST implement BIP-39 exactly:

- mnemonic and passphrase are UTF-8 NFKD normalized;
- salt is UTF-8 NFKD “mnemonic” || passphrase;
- PBKDF2-HMAC-SHA512 uses 2,048 iterations;
- output is a 64-byte seed.

BIP-39 derives only the wallet master seed. Quantum child keys MUST then use a separately reviewed, domain-separated post-quantum derivation profile. Wallet storage encryption MUST have its own memory-hard password-KDF profile and MUST not describe the BIP-39 PBKDF as storage protection.

5. Private note and transaction model

5.1 Note model

A note is private witness data:

```
Note {
    version
    asset_id
    value_u64
    spend_authorization_key_digest
    nullifier_key_digest
    recipient_data
    commitment_randomness
}
```

Public state contains a note commitment, not the plaintext. A recipient learns the note through an authenticated encrypted payload. The encrypted payload MUST bind to the commitment, transaction identifier, output index, chain identifier, and protocol version.

5.2 Public transaction fields

The public transaction contains only:

- protocol version and chain identifier;
- one approved anchor root and its consensus context;
- a bounded vector of nullifiers;
- a bounded vector of new note commitments;
- corresponding encrypted-note payloads or authenticated payload digests;
- public fee in base units;
- expiry/finality context;
- one active authorisation profile identifier; and
- proof mode and proof bytes or aggregate-proof reference.

Every vector length and byte string MUST have a versioned maximum before parser implementation. Parsers MUST reject overlong, truncated, duplicate, non-canonical, unknown-version, unknown-authorisation-profile, and trailing-byte encodings before expensive cryptographic work.

5.3 Private witness

For every input, the witness contains the full note plaintext, commitment randomness, membership path, nullifier secret, authorization public key, and profile-specific authorization witness. Stateful signing state remains in the wallet and is not part of the transaction proof witness. For every output, the witness contains the full new note plaintext, commitment randomness, and encryption witness required to bind the public encrypted payload.

Input commitments and their openings are mandatory witness relations. They MUST NOT be omitted merely because only the anchor root is public.

5.4 Transaction validity relation

A verifier accepts only if the proof establishes all of the following:

1. **Canonical context**: all public and private encodings use the selected version, chain, asset, and domain tags.

2. **Input opening**: for each input i, Commit(pp, input_note[i], input_rho[i]) equals the commitment leaf authenticated by its membership path.
3. **Membership**: every input leaf is a member of the public anchor root under the exact tree hash and depth.
4. **Nullifier correctness**: each public nullifier is derived from the opened note, its bound nullifier key, and the chain/version domain.
5. **Authorization**: the opened note binds the authorization key, the declared active profile verifies its complete witness over the exact transaction authorization digest, and every input uses that same profile.
6. **Output opening**: for each output j, Commit(pp, output_note[j], output_rho[j]) equals the corresponding public output commitment.
7. **Encryption binding**: every public encrypted note or digest authenticates the same output plaintext and output position.
8. **Range**: every value and fee is a canonical unsigned 64-bit base-unit integer and obeys per-note and asset supply bounds.
9. **Integer conservation**: sum(input_values) = sum(output_values) + fee as an integer, except in the separately typed reward transaction.
10. **No local duplicates**: input nullifiers and output commitments are unique within the transaction.
11. **Public consistency**: counts, indices, flags, expiry, authorisation profile, and proof mode agree across the signed digest, encrypted payloads, and proof.

The verifier then performs state checks outside the proof: the anchor is currently admissible, each nullifier is globally unused, the transaction is not expired, resource limits hold, and the proof version is active.

5.5 Carry-safe balance constraints

Field equality is insufficient because field arithmetic can wrap. Each u64 value MUST be decomposed into four 16-bit limbs, with a range constraint for every limb. The circuit MUST:

1. add all input values into an accumulator wide enough for the maximum input count, constraining every base-2^16 carry;
2. add all output values and the fee into an equally wide accumulator, constraining every carry;
3. constrain every accumulator limb and final carry equal;
4. reject overflow beyond the protocol-wide maximum sum.

The maximum vector length determines the accumulator width. This relation is over integers represented in the proof field; no unchecked field reduction is allowed. The 16-bit limb representation is an arithmetic encoding, not a claim that a commitment is homomorphic across carries.

5.6 Commitment tree

The candidate note tree depth is 64, subject to benchmark and state-layout review. MERKLE_HASH_BYTES is deliberately unresolved: T001, T201, and T302 MUST derive it from the concrete quantum collision, second-preimage, multi-target, and protocol-lifetime analysis. Empty nodes are then defined recursively:

```
empty[0] = QH("empty-leaf", empty, MERKLE_HASH_BYTES)
empty[d + 1] = QH(
    "merkle-node",
    empty[d] || empty[d],
    MERKLE_HASH_BYTES
)
```

Leaves and internal nodes MUST use different tags. Insertion order MUST come from the canonical DAG state order. An anchor-acceptance window MUST be derived from measured

proof-generation latency, propagation, finality, and reorg risk; a fixed "last 100 headers" rule is not acceptable without that derivation.

5.7 Selective disclosure capabilities

Selective disclosure is a wallet capability, not a consensus bypass. The initial profile MUST specify non-overlapping key or capability types for:

- **incoming view**: discover and decrypt received notes without spend authority;
- **full-wallet view**: reconstruct the explicitly authorised incoming and outgoing wallet history without spend authority;
- **transaction disclosure**: reveal one transaction's selected plaintext and cryptographic binding without exposing unrelated wallet history; and
- **auditor-scoped disclosure**: derive a capability restricted by a canonical scope such as account branch, counterparty set, transaction class, or time interval.

Each exported capability MUST state exactly what it reveals, what it cannot prove, its scope encoding, and its forward/backward visibility. Wallets MUST warn that data already disclosed cannot be cryptographically retracted. Revocation MAY stop future access only where the underlying construction supports it; it MUST NOT be described as erasing copied history. No consensus participant, operator, foundation, or regulator receives a universal recovery or viewing key by design.

Disclosure receipts or proofs MAY support an external audit workflow, but they do not themselves establish legal compliance, identity, source of funds, or an absence of undisclosed transactions.

5.8 Bounded payment policies and interoperability boundary

The candidate application surface is a versioned, finite set of private payment policies rather than arbitrary bytecode. Candidate policies include absolute/relative timelocks, hashlocks, threshold or multisignature authorisation, escrow, recurring-payment authorisations, atomic-swap conditions, and conditional release. Every enabled policy MUST have:

- one canonical encoding and explicit state-transition semantics;
- bounded execution, witness, proof, verification, and storage costs;
- a validity relation that preserves note privacy and exact supply;
- domain-separated authorisation and replay protection;
- positive, negative, timeout, reorg, and recovery vectors; and
- independent review under the same release gates as ordinary transfers.

Unknown policy identifiers and unsupported versions MUST fail closed. There is no fallback interpreter and no implicit general-purpose virtual machine. Private asset issuance is deferred until an asset-specific conservation, authorisation, disclosure, and lifecycle profile passes a separate release decision; asset_id in the note schema reserves domain separation but does not claim that such issuance exists.

Cross-chain atomic swaps or adapters MAY be researched after the core transaction feasibility gate. External consensus, custody, multisignature/MPC, oracle, light-client, and bridge assumptions remain outside Quantum's base security claim. An adapter MUST NOT be described as post-quantum secure, trustless, private, or production-ready merely because the Quantum side meets its own gates.

6. Serialization and identifiers

Consensus serialization MUST be specified field by field before implementation:

- fixed little-endian integers of explicit width;
- length prefixes of explicit width followed by bounded data;

- no floating-point values;
- no implicit maps, platform-dependent structs, Unicode identifiers, or alternate encodings;
- shortest/canonical form only;
- dedicated tags for transaction ID, authorization digest, note commitment, nullifier, Merkle leaf/node, block ID, PoW, KDF, and address checksum.

The candidate human-readable address is:

```
"qtm_" || lower_base32(version || network || payload || checksum)
```

lower_base32 uses the RFC 4648 alphabet abcdefghijklmnopqrstuvwxyz234567, lower case only, without padding. The checksum is the first eight bytes of QH("address-checksum", version || network || payload, 32). The final profile MUST first fix the payload and maximum length; until then, addresses are research-only and MUST NOT be used to receive value.

7. DAG consensus and state

7.1 Consensus profile — blocking gate

Quantum evaluates proof-of-work GHOSTDAG, whose research basis includes Sompolinsky, Wyborski and Zohar. The paper name is not a complete executable specification. Before implementation, a versioned consensus profile MUST pin:

- parent selection and well-formedness;
- anticone parameter and blue/red classification;
- selected-parent chain and merge-set ordering;
- accumulated work/blue-work calculation;
- finality and pruning rules;
- timestamp validity;
- block weight and data-availability limits;
- network-delay and adversarial-work assumptions.

"Highest blue score wins" and sorting blocks by score are not sufficient definitions.

7.2 Header and proof of work

The final header layout MUST have one canonical byte encoding and one block-hash function. At minimum it binds version, network/chain ID, parent set, transaction root, pre-state root, post-state root, timestamp, nonce, claimed target, consensus score/work commitments, and extension commitment.

Block validation MUST:

1. derive the canonical past from the DAG profile;
2. recompute the expected target using deterministic integer arithmetic;
3. require the header target to equal that expected target;
4. hash the canonical header under the PoW domain;
5. require the hash integer to be at most the expected target;
6. validate timestamp bounds from consensus history, using local time only for a non-consensus future-admission check.

An unverified miner-supplied target would make mining trivial and is forbidden.

7.3 Canonical state transition

The selected-parent chain and ordered merge set MUST produce one deterministic transaction sequence. Starting from the canonical pre-state, validators apply transactions sequentially and atomically:

1. reject duplicate transaction identifiers;
2. verify proof and public format limits;

3. verify anchor admissibility;
4. reject any nullifier already in state or earlier in the same ordered batch;
5. append output commitments in order;
6. update nullifier and commitment roots;
7. require the computed post-state root to equal the header commitment.

For concurrent spends, the first transaction in canonical order may succeed; later uses of the same nullifier MUST fail. The design requires a proof that all honest nodes derive the same result across reorgs, pruning, and recovery.

7.4 Difficulty adjustment

The previous wall-clock/float pseudocode is withdrawn. The final DAA MUST be specified in exact integer equations over canonical DAG history, with clamping, overflow behavior, timestamp manipulation analysis, test vectors, and cross-implementation tests. A node's current time MUST NOT alter the expected target for an already received DAG.

7.5 Rewards, supply, and genesis

The monetary target is a hard cap of 21,000,000 QTM with no premine, ICO allocation, or founder reward. The exact emission curve remains provisional.

A reward transaction MUST be a distinct consensus type. Its permitted subsidy MUST be calculated from a canonical DAA score or other explicitly defined DAG measure—not an ambiguous linear block height—and MUST specify eligibility for blue, red, merged, and stale blocks.

For each canonical reward transition, validators MUST derive exact integers:

```
collected_fees = sum(fee of each accepted ordinary transaction)
reward_outputs <= authorized_subsidy + collected_fees
claimed_subsidy = max(0, reward_outputs - collected_fees)
claimed_subsidy <= authorized_subsidy
next_cumulative_issuance = cumulative_issuance + claimed_subsidy
next_note_supply = previous_note_supply
                   - collected_fees
                   + reward_outputs
```

The authorized subsidy MUST be non-negative and no greater than 21,000,000 QTM - cumulative_issuance. The fee portion is transferred value and MUST NOT increment cumulative issuance; only the net increase in note supply may be counted as claimed subsidy. Any difference between authorized_subsidy + collected_fees and reward_outputs is burned. Reward outputs and fee claims MUST use the same private note system without exposing recipient data beyond the final consensus disclosure budget. The profile MUST define reward maturity, reorg reversal, eligibility, and atomic application so that a fee or subsidy cannot be claimed twice. Supply accounting MUST prove these equations and the cap under rounding and reorganization.

There is no canonical genesis block at this revision. Before a network launch, one immutable genesis byte string, timestamp unit, message field, target, and hash algorithm MUST be published with vectors. Seconds and milliseconds MUST not be mixed, and genesis MUST use the same block hashing rules as later blocks.

8. Network anonymity and transport

The P2P design has two separate obligations:

1. **link security**: post-quantum authenticated, replay-protected, downgrade-resistant channels; and
2. **origin privacy**: resistance to mapping a transaction to its source.

The link profile may combine ML-KEM-1024, SLH-DSA, a transcript KDF, and a 256-bit authenticated cipher only after a reviewed composition fixes every message and failure path. Authentication MUST not create a stable user identity or link wallet activity.

The origin-privacy profile MUST evaluate stem/fluff diffusion, mix or onion routing, cover traffic, delayed batching, peer rotation, route failures, eclipse resistance, and active tagging. Dandelion++ is relevant research (paper) but promises probabilistic de-correlation under a model, not universal prevention of correlation.

Release evidence MUST include network simulation, adversarial experiments, privacy metrics with confidence intervals, and independent review. Simple byte uniformity or Pearson-correlation tests are diagnostics, not anonymity proofs.

9. Scalability and resource budget

9.1 End-to-end target

The 1,000 transactions-per-second figure is an acceptance target, not a current capability claim. A benchmark passes only when accepted state transitions—not submitted requests—sustain the target while R1–R5 remain enabled.

The reference workload MUST include a published mix of input/output counts, note scans, conflicts, reorgs, proof modes, peer delays, and malformed traffic. Results MUST report at least p50/p95/p99 latency and resource use.

9.2 Feasibility budget

At 1 Gbit/s, a link has a theoretical 125 MB/s before protocol overhead. With 20% headroom and four outgoing gossip copies, only about 25 MB/s remains for unique transaction data: roughly 25 KB per transaction at 1,000 tx/s. A 110 KB transaction therefore cannot meet that example topology, even before headers and retransmission.

Likewise, 25 KB $\times$ 1,000 tx/s is roughly 788 TB/year. A 4 TB operational-node target is possible only with compact proofs, pruning/state commitments, and a separate archival/recovery design. These calculations are mandatory design inputs, not optional optimizations.

The final profile MUST publish budgets for:

- average and worst-case wire transaction size;
- proof size and aggregate amortization;
- verifier operations and memory per transaction;
- inbound/outbound propagation amplification;
- nullifier, commitment, block, and archive growth;
- wallet scan bandwidth and time;
- snapshot size, creation time, verification time, and recovery trust.

9.3 Pre-node cryptographic feasibility gate

Before full-node or GHOSTDAG integration begins, two independent implementations MUST execute a representative private transaction with exactly two inputs and two outputs for every retained authorisation arm. Apart from authorisation and its required state, the statement, witness relation, proof profile, hardware, and measurement method MUST remain identical. The measured relation MUST include:

- both input commitment openings and Merkle membership paths;
- nullifier derivation and uniqueness constraints;
- both output openings and encrypted-note binding;
- complete authorisation verification inside the proof for the declared profile, including exact SLH-DSA-SHAKE-256f for the incumbent arm;
- 64-bit ranges, carry-safe integer conservation, and the public fee; and

- the selected STARK transcript, zero-knowledge masking, verifier, and, if proposed, aggregation path.

The non-normative T305 research protocol pre-registers the literature boundary, exact experiment, measurement fields, independent-implementation rules, and stop/go evidence for this gate. It may make the experiment more specific but cannot omit or weaken a requirement in this specification. The versioned T305 prior-art and reuse decision records why the experiment is comparative, which public work must be replicated or labelled author-reported, and which code may not be adopted. Named T001 owner and reviewer signatures remain mandatory before implementation.

T004 MUST freeze reference desktop and constrained-client hardware, workload, parallelism, proof-latency, memory, verifier, proof-size, and aggregate amortisation thresholds before results are interpreted. The report MUST publish single-wallet latency separately from aggregate throughput; multiplying ideal parallel jobs is not a wallet-latency result.

If the incumbent stateless arm passes and meets its frozen material-benefit rule, it may proceed. If a reviewed stateful arm passes while the incumbent fails or provides no sufficient benefit, the normative authorisation profile MUST be revised before integration. If no arm meets the frozen feasibility budget, that is a design failure, not a node optimisation backlog. The signature profile, commitment, validity relation, proof system, or explicit system requirements MUST be revised, or the project MUST stop before consensus integration. No fabricated "hundreds" or "thousands" of proofs per second threshold substitutes for the published hardware and end-to-end budget.

9.4 End-to-end performance gates

Before a public scalability claim:

- the exact benchmark revision and toolchain are pinned;
- the same consensus parser and verifier used by nodes are measured;
- at least two independent implementations interoperate;
- a 24-hour target run and a longer stability run complete without state-root divergence;
- overload behavior remains bounded and malformed inputs are rejected before disproportionate work;
- raw artifacts and reproduction commands are published.

10. Assurance and release gates

10.1 Required analytical artifacts

1. complete protocol and threat model;
2. commitment construction with concrete post-quantum security analysis;
3. STARK soundness and zero-knowledge analysis, including QROM composition;
4. transaction validity proof covering both input and output openings;
5. GHOSTDAG/state-ordering safety and liveness analysis;
6. DAA and issuance correctness proof;
7. network anonymity analysis;
8. end-to-end multi-target security budget.

10.2 Required implementation evidence

1. pinned toolchains and reproducible builds;
2. official KATs for FIPS 202, FIPS 203, and FIPS 205;
3. canonical cross-language vectors for every Quantum encoding;
4. differential tests between two independent implementations;
5. fuzzing, property tests, malformed-input and resource-exhaustion tests;
6. constant-time and side-channel review for secret-dependent operations;
7. state/reorg/recovery model checking or formal verification where feasible;

8. reproducible performance and anonymity experiments.

10.3 Independent review

Testnet promotion requires named human owners and independent specialist review for cryptography, proof systems, consensus, networking, implementation security, and economics. Production requires closure or explicit rejection of every high-severity finding and a public statement of residual risk.

An AI system may assist with implementation or analysis, but MUST NOT approve its own cryptographic design, proof, audit, benchmark, or release.

10.4 Go/no-go matrix

Gate	Pass condition	Current state
G1 Requirements	R1–R8 traceable to tasks and tests	Drafted, not independently reviewed
G2 Commitment	Exact scheme and 128-bit composed PQ analysis	Blocked
G3 Proof	Complete AIR/transcript/ZK/soundness profile	Blocked
G3A Transaction feasibility	Complete 2-input/2-output proof meets frozen client, verifier, size, and aggregation budgets	Not run
G4 Private transaction	No-inflation and authorization relation reviewed	Draft relation only
G5 DAG state	Deterministic consensus and conflict handling proved/tested	Blocked
G6 Network anonymity	Threat model, design, experiments, review pass	Blocked
G7 Scalability	1,000 accepted tx/s end to end with artifacts	Not run
G8 Interoperability	Two independent implementations and vectors	Not started
G9 External audit	Critical/high findings resolved	Not started

No "specification complete," "quantum-secure," "fully anonymous," or "1,000 TPS achieved" claim is permitted while the corresponding gate is open.

11. Legal and licensing boundary

Repository-authored research text is dedicated under CC0-1.0 as described in the repository LICENSE file. CC0 does not grant patent or trademark rights and does not make a design "patent-free." Third-party standards, papers, names, and implementations retain their own rights and licenses.

This document is technical research, not legal, financial, tax, or investment advice. It is not an offer to sell a token, security, or network service. Privacy features do not remove obligations under applicable law. Obtain specialist legal advice before any launch or token distribution.

12. References

Primary references:

1. NIST, FIPS 202: SHA-3 Standard.

2. NIST, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.
3. NIST, FIPS 205: Stateless Hash-Based Digital Signature Standard.
4. NIST, SP 800-208: Recommendation for Stateful Hash-Based Signature Schemes.
5. NIST, SP 800-230 initial public draft: Recommendations for Parameter Sets of HSS, XMSS, and SLH-DSA, 2026; non-final at this revision.
6. NIST, Post-Quantum Cryptography project and standardization status.
7. TzEL contributors, TzEL whitepaper,

2026.

8. Alupotha, Boyen and McKague, LACT+: Efficient Lattice-Based Aggregatable Confidential Transactions, 2023.
9. Ben-Sasson et al., Scalable, transparent, and post-quantum secure computational integrity.
10. Ben-Sasson et al., DEEP-FRI.
11. Baum et al., More efficient commitments from structured lattice assumptions.
12. Sompolinsky, Wyborski and Zohar, PHANTOM/GHOSTDAG.
13. Fanti et al., Dandelion++.
14. Noise Protocol Framework.
15. Bitcoin BIPs, BIP-39.
16. Hosoyamada and Sasaki, Finding Hash Collisions with Quantum Computers.
17. Béguinet et al., GeT a CAKE: Generic Transformation from KEM to PAKE.

Appendix A — Decisions deliberately not frozen

The following are intentionally unresolved because choosing numbers without analysis would create false precision:

- the commitment construction and parameters;
- the proof field extension, AIR, FRI, masking, and aggregation profile;
- transaction and vector byte maxima;
- note-encryption and P2P authenticated-channel composition;
- address payload and derivation hierarchy;
- GHOSTDAG anticone/finality parameters and DAA;
- reward curve and genesis block;
- anonymity network parameters;
- reference hardware and latency thresholds beyond the 1,000 tx/s acceptance target.

Each item has an owner task in the verification guide. A value becomes normative only with rationale, vectors, tests, and review.

Appendix B — Revision record

0.4.0-research — 2026-07-23

- recorded TzEL as the closest public note/nullifier/ML-KEM/hash-authorisation/STARK engineering baseline and rejected a generic protocol novelty claim;
- changed FIPS 205 SLH-DSA-SHAKE-256f from an assumed final profile to the stateless incumbent in a pre-registered comparison;
- required independently specified TzEL-shaped and applicable NIST SP 800-208 stateful comparators under the same complete transaction relation;
- made state-management failure analysis and a material-benefit rule mandatory before retaining the stateless profile;
- linked the versioned T305 prior-art and reuse decision, required named sign-off before implementation, and prohibited unlicensed source-code adoption.

0.3.0-research — 2026-07-21

- narrowed the initial product boundary to private post-quantum cash and settlement rather than a general-purpose smart-contract platform;
- added a mandatory complete 2-input/2-output proving feasibility gate before wallet, node, or DAG integration;
- separated wallet transaction proofs from block aggregation and required an aggregate to replace, not duplicate, individual proofs on the consensus wire;
- defined incoming, full-wallet, transaction-specific, and auditor-scoped disclosure capabilities without a universal viewing backdoor;
- introduced a finite, proof-bounded payment-policy direction and made unknown policy versions fail closed;
- made private assets and cross-chain adapters separate post-core profiles whose external trust assumptions do not inherit Quantum's base claims;
- recorded the product position as a private post-quantum settlement research path for people, organisations, and autonomous software agents.

0.2.0-research — 2026-07-09

- converted the document from a claimed complete formal specification to an evidence-gated research design;
- retained post-quantum security, default anonymity, and 1,000 layer-1 TPS as hard release requirements;
- closed the specification-level inflation gap by binding every input and output value to a commitment opening and requiring carry-safe integer conservation;
- removed the unsupported commitment formula, biased sampler, incorrect homomorphism claim, and incomplete STARK security arithmetic;
- replaced SPHINCS+ protocol naming with FIPS 205 SLH-DSA;
- removed the invented post-quantum Noise pattern and separated link security from origin anonymity;
- made target validation, deterministic DAA, DAG state ordering, rewards, and genesis explicit blocking deliverables;
- added realistic bandwidth/storage feasibility budgets and proof-aggregation semantics;
- corrected BIP-39 derivation and CC0 patent/trademark wording.
- made consensus digest lengths conditional on concrete quantum collision, second-preimage, multi-target, and lifetime analysis;
- added an explicit recipient-key-privacy requirement for encrypted notes;
- defined fee transfer, claimed subsidy, burns, reward maturity, and cumulative issuance as one state-transition invariant.